Scaffold (F26-074) — Panel Feedback Response Document

Status: Approved With Major Modifications (No redefense) **Action:** This document addresses every concern raised by the panel, point by point, with the technical specificity requested.

1. "How does Deep Knowledge Tracing work?"

The panel asked this because the team could not explain it clearly during the defense. Here is the explanation that should have been given, and that will be incorporated into the revised proposal.

BKT (Bayesian Knowledge Tracing) — the baseline

BKT is a Hidden Markov Model with two hidden states per skill: "mastered" and "not mastered." For each student-skill pair, it maintains four probabilities:

- $P(L_0)$ — probability the student already knew the skill before any practice
- P(T) — probability the student transitions from "not mastered" to "mastered" after one practice opportunity
- P(G) — probability the student guesses correctly despite not having mastered the skill
- P(S) — probability the student slips (answers incorrectly despite having mastered the skill)

After each student submission (correct or incorrect), BKT updates

P(mastered) using Bayes' rule:

```
If the student answered correctly:
  P(mastered | correct) = P(correct | mastered) × P(maste
  where P(correct | mastered) = 1 − P(S)
  and P(correct) = P(mastered) × (1 − P(S)) + (1 − P(mast

Then the learning update:
  P(mastered_new) = P(mastered | observed) + (1 − P(maste
```

BKT treats each skill independently. It cannot capture that a student who masters "loops" is more likely to understand "arrays" than a student who hasn't.

DKT (Deep Knowledge Tracing) — the primary model

DKT replaces the explicit Bayesian update with a recurrent neural network (LSTM). The key insight: instead of tracking each skill separately with four fixed parameters, the LSTM learns a hidden state vector that implicitly encodes the student's knowledge across all skills simultaneously.

Input at each timestep: A one-hot encoded vector combining the skill attempted and whether the answer was correct. For example, if there are 10 skills, the input is a 20-dimensional vector (10 skills × 2 outcomes). If the student attempted skill 3 and got it correct, position 3 is set to 1 (all others 0).

What the LSTM does: It processes the sequence of these input vectors one at a time, maintaining a hidden state h_t that evolves with each new interaction:

```
h_t = LSTM(x_t, h_{t-1})
```

This hidden state h_t is a dense vector (typically 100–200 dimensions) that encodes everything the model has learned about this student's knowledge from their entire interaction history up to time t.

Output: A fully connected layer maps h_t to a vector of probabilities, one per skill, predicting the probability the student will answer correctly on each skill at the next opportunity:

```
y_t = sigmoid(W × h_t + b)
```

So y_t[5] = 0.82 means the model predicts an 82% chance the student will correctly solve a problem involving skill 5 on their next attempt.

Why DKT is better than BKT for our use case: The hidden state h_t captures cross-skill dependencies. If the model observes that students who master "loops" tend to also improve on "arrays," the LSTM learns this relationship from the training data. BKT cannot do this — it tracks each skill in isolation.

Training: DKT is trained on historical student interaction logs (sequences of skill attempts and outcomes) using standard backpropagation through time (BPTT), minimizing binary cross-entropy loss between predicted and actual correctness.

How Scaffold uses the DKT output

The DKT's output vector y_t is our "mastery vector." For a course with 12 knowledge components, it might look like:

```
{
  arrays: 0.91,
  loops: 0.87,
  conditionals: 0.82,
  functions: 0.74,
  pointers: 0.45,
  recursion: 0.23,
  linked_lists: 0.18,
  stacks: 0.55,
  queues: 0.52,
  trees: 0.08,
  sorting: 0.31,
  hashing: 0.05
```

```
}
```

This vector is injected into the LLM's prompt context so the hint generator knows: this student understands arrays and loops well but struggles significantly with recursion and has almost no exposure to trees. A hint about a tree traversal problem should therefore explain recursion concepts (which the student is weak on) rather than assuming they understand recursive function calls.

---

2. "If a student has no prior learning history, how will you evaluate them using DKT?" (The Cold-Start Problem)

This is a legitimate and well-known problem in knowledge tracing called the "cold-start problem." The panel is right to ask about it. Here is our three-layer mitigation strategy:

Layer 1: Population prior initialization

When a brand-new student enters the system with zero interaction history, we initialize their mastery vector using the average mastery vector computed from the training data. If the model was trained on 800 students from the CSEDM dataset, the new student starts with the average initial mastery of those 800 students. This is not a guess — it is a Bayesian prior derived from observed data.

Concretely, the DKT's hidden state h_0 is initialized to the mean initial hidden state across all training students, rather than a zero vector.

Layer 2: Diagnostic quiz at the start of the course

Before the first lab, each student completes a short diagnostic assessment (5–8 questions, 10–15 minutes) covering the prerequisite

concepts for the course. These questions are tagged with knowledge components. The results are fed into the DKT as the student's first few interaction data points, rapidly personalizing the mastery vector before the student writes their first line of lab code.

For example, if the diagnostic reveals a student already understands loops and conditionals but fails the recursion question, the DKT's hidden state after processing these 5–8 interactions will already reflect: high mastery on loops/conditionals, low mastery on recursion. The student's first lab hint will be calibrated accordingly.

Layer 3: Rapid adaptation from early interactions

DKT's hidden state updates after every single interaction. Within the first 10–15 submissions of Lab 1, the model has enough signal to substantially differentiate one student from another. A student who gets the first three array problems correct but fails the first pointer problem will have a meaningfully different mastery vector from a student who struggles with arrays from the start.

The key insight: cold-start is a problem for the first 15–30 minutes of the first lab only. After that, every student has a personalized profile. This transient cold-start period is handled by the diagnostic quiz (Layer 2) and the population prior (Layer 1).

What happens during cold-start if a hint is needed?

During the cold-start window, the system defaults to conservative behavior: it uses a mid-level disclosure policy (Level 1–2 hints: conceptual nudges and strategy direction) rather than extreme levels. As the mastery vector gains confidence from real interactions, the system's hint calibration becomes more precise. This degradation is graceful — the worst case during cold-start is that the hint is "one-size-fits-all" for a few minutes, which is exactly what every existing system does all the time.

---

3. "Clearly define what 'leakage' means and how you will identify, measure, and evaluate it"

This was the panel's strongest criticism and it is entirely valid. The previous proposal used "leakage" informally without quantification. Here is the formal definition.

Definition of code leakage

Code leakage is the unintended disclosure, by the hint generation system, of information that allows a student to produce a correct solution without understanding the underlying concept. In the code domain, leakage is not binary — it exists on a graduated spectrum measured by structural similarity between the hint's content and the reference solution.

The Code Disclosure Taxonomy (5 levels)

Level	Name	Description	Example (for a linked list reversal problem)	AST similarity to solution
L0	Conceptual nudge	A question or observation about the concept, containing no code, no pseudocode, no structural hints	"What should happen when you reach the end of the list?"	0.00

L1	Strategy direction	Names the algorithmic approach without showing structure	"You need to track multiple pointers as you traverse"	0.00–0.05
L2	Targeted concept explanation	Explains the relevant concept with a generic example unrelated to the current problem	"In general, reversing a sequence involves changing where each element points to — think about what 'next' should become"	0.05–0.15
L3	Pseudocode or skeleton with gaps	Shows the structure of the solution with critical pieces missing	"while (current != NULL) { next = ___; current->next = ___; prev = ___; current = ___; }"	0.30–0.60
L4	Near-complete code	Provides most of the solution with only a trivial piece missing (a variable name, a return statement)	The full function with only the return statement replaced by "// return what?"	0.70–0.95

How leakage is measured: the Leakage Score

For every hint the system generates, we compute a **Leakage Score**

(LS) between 0.0 and 1.0:

```
LS(hint, solution) = AST_similarity(extract_code(hint), p
```

Where:

- `extract_code(hint)` parses the hint text and extracts any code fragments (using regex for code blocks + a syntax checker for inline code patterns)
- `parse(solution)` produces the AST of the reference solution
- `AST_similarity` computes normalized tree-edit distance between the two ASTs

If the hint contains no parseable code, LS = 0.0. If the hint contains the complete solution verbatim, LS = 1.0.

AST similarity implementation: We use the `tree-sitter` library (available for C, C++, Python, and 40+ languages) to parse both the hint's code content and the reference solution into ASTs. We then compute the Zhang-Shasha tree-edit distance, normalized by the size of the reference solution's AST:

```
AST_similarity = 1 - (tree_edit_distance(AST_hint, AST_so
```

This gives 0.0 for completely different trees and 1.0 for identical trees.

Leakage thresholds and system behavior

Leakage Score range	Classification	System action
0.00–0.15	No leakage	Hint delivered to student
0.15–0.40	Acceptable partial disclosure	Hint delivered; logged for review

0.40–0.60	Warning zone	Hint regenerated with a stricter disclosure-level instruction; if regenerated hint still exceeds 0.40, deliver with a log entry
0.60–1.00	Leakage detected	Hint blocked; regenerated at one disclosure level lower; incident logged; if 3+ leakage incidents in one session, alert the instructor

Leakage metrics reported in the thesis

The following metrics will be computed and reported for each experimental condition:

1. **Leakage rate:** Percentage of all generated hints with LS > 0.40 (the warning threshold)
2. **Mean leakage score:** Average LS across all generated hints
3. **Leakage-per-disclosure-level:** Mean LS broken down by intended disclosure level (L0–L4), to verify that the system's intended level matches its actual output
4. **Jailbreak-induced leakage rate:** Percentage of adversarial student prompts (detected by the jailbreak classifier) that result in LS > 0.40 despite the guardrail
5. **Guardrail catch rate:** Percentage of hints with LS > 0.40 that were caught and regenerated before reaching the student

How leakage evaluation differs between Condition B and Condition C

- **Condition B (prompt-based guardrail):** Leakage prevention relies on prompt instructions ("do not provide code") plus the output AST-similarity filter
- **Condition C (DPO-aligned model):** Leakage prevention is baked into the model weights via preference training, plus the same output filter

The comparison between Conditions B and C directly answers: does

model-level alignment produce a lower leakage rate than prompt-level guardrails? The AST-similarity metric makes this comparison quantitative, not subjective.

---

4. "What is the actual computing challenge and technical contribution?"

The panel felt this was not explained clearly. Here is the precise statement.

The computing challenge (what is technically hard)

The computing challenge is building a **real-time, closed-loop adaptive system** that connects three independently studied research areas — IDE telemetry, knowledge tracing, and LLM hint generation — into a single pipeline that operates during a live programming lab with 40–60 concurrent students.

This is hard because:

Real-time constraint. The pipeline must process a student's telemetry event, update their knowledge state, detect if they are struggling, generate a hint conditioned on their mastery vector, filter it for leakage, and display it — all within 8 seconds. This is not a batch job. It runs continuously for 60 students simultaneously.

The KT-to-LLM bridge has never been built. No published system takes the output of a knowledge tracing model and injects it into an LLM's context for hint generation. This bridge — converting a numeric mastery vector into natural language context that meaningfully changes the LLM's hint — is an engineering and prompt-design problem that does not exist in the literature. Every KT paper produces a number. Every LLM tutoring paper ignores that number. We connect them.

Code leakage is harder than math leakage. All existing pedagogical alignment work (TutorRL, PEARL, SocraticLM) operates on math, where

leakage is binary: the answer "42" was or was not revealed. In code, leakage is a spectrum (function signature → pseudocode → partial code → full solution), requiring a novel graduated metric (the AST-similarity Leakage Score defined in Section 3 above) and a corresponding graduated guardrail policy. This metric does not exist in the literature.

The technical contribution (what is new)

1. **The closed loop itself.** First system integrating real-time IDE telemetry → automated struggle detection → knowledge tracing → knowledge-state-conditioned LLM hint generation → graduated code leakage guardrail, deployed and evaluated in a live classroom.

2. **The KT-conditioned prompting methodology.** A structured prompt construction pipeline that injects a per-student, per-concept mastery vector (from DKT) into the LLM's context, with a disclosure-level policy function that selects hint specificity based on mastery score, struggle duration, and hint history. This prompting methodology can be reused with any KT model and any LLM — it is not tied to a specific model.

3. **The graduated Code Leakage Score.** A quantitative metric (AST-similarity-based, 0.0–1.0) for measuring code disclosure in LLM tutoring responses, with defined thresholds and automated enforcement. This metric generalizes the binary leakage concept from math tutoring to the code domain.

4. **Empirical evaluation.** A three-condition controlled classroom study comparing (a) reactive LLM assistance without KT, (b) KT-conditioned proactive assistance with prompt guardrails, and (c) KT-conditioned proactive assistance with DPO-aligned guardrails. The primary outcome — transfer-task score — directly measures whether the system improves actual learning, not just task completion.

---

5. "Define your features across the problem;

---

do not include unnecessary features"

The panel felt the proposal included too many features without clear justification for each. Here is the refined, prioritized feature set with explicit justification for why each is necessary and what would break without it.

Core features (must-have for the contribution to hold)

Feature	Why it is necessary	What breaks without it
Knowledge tracing (BKT → DKT)	This is the core of the contribution — the system must know what the student has and hasn't mastered	Without KT, hints are generic (one-size-fits-all), which is the baseline condition, not the contribution
KT-conditioned hint generation	The mastery vector must actually change the hint the student receives	Without this, the KT model produces numbers that nobody uses — the same gap we criticize in existing KT papers
Code leakage guardrail with AST-similarity metric	Without measurable leakage prevention, the system might just give away solutions	The system becomes a code generator, not a tutor — defeats the entire purpose
Classroom study (Condition A vs Condition B)	Without evaluation, there is no evidence the system works	The project becomes pure engineering with no research contribution

Important features (strongly support the contribution)

Feature	Why it is important	Can the project survive without it?

Struggle detection from telemetry	Enables proactive intervention — a key differentiator from all existing systems	Yes, but the system becomes reactive only (student must ask for help), losing a major advantage
DPO alignment experiment (Condition C)	Tests whether model-level guardrails outperform prompt-level guardrails	Yes — this is the secondary contribution; the primary study (A vs B) stands alone
Instructor dashboard	Gives instructors visibility into student progress and system behavior	Yes — the student-facing system works without it, but instructor buy-in for the study depends on it

Removed / deferred features (cut from scope based on panel feedback)

Feature	Why it is being cut	Where it goes
Cross-session persistent profiles	Adds complexity without being essential to the core research question	Future work section of the thesis
Copy-paste source tracking	Interesting but not necessary for the leakage or KT contribution	Future work
Full GRPO/RL training (Track B)	Requires multi-GPU infrastructure not confirmed; DPO on single GPU is sufficient	Explicitly out of scope
Post-lab PDF report generation for students	Nice-to-have UI feature, not a computing contribution	Future work

This brings the feature set down to **4 core features + 3 important features = 7 total**, each with a clear reason for inclusion.

---

6. "The way students were explaining is wrong" and "Quantify them"

The panel's feedback suggests the team explained the project in narrative/hand-waving terms rather than precise technical terms. The revised explanation must quantify every claim. Here is a mapping from what was probably said versus what should have been said:

What was probably said	What should have been said
"We detect if a student is struggling"	"The struggle detector classifies the student's state into one of four categories (flow, productive struggle, unproductive struggle, copying) based on four telemetry signals: repeated same-class compilation errors (≥3 in 5 minutes), edit thrashing (edit distance between saves < 15 characters with >5 saves), idle time (>4 minutes without meaningful edit), and code regression (test cases that previously passed now failing). Target: ≥0.75 precision, ≥0.70 recall on the unproductive-struggle class."
"The AI gives hints instead of solutions"	"The hint engine generates responses at one of five disclosure levels (L0–L4). Each generated response is scored with our Leakage Score metric (AST-similarity between the response's code content and the reference solution, range 0.0–1.0). Responses with LS > 0.40 are automatically regenerated at a lower disclosure level. Target leakage rate: <5% of all hints with LS > 0.40."

"We use Deep Knowledge Tracing"	"DKT is an LSTM-based sequence model trained on student interaction logs. Input: one-hot encoded (skill_id, correct/incorrect) per timestep. Output: predicted P(correct) for each skill at the next opportunity. Hidden state dimension: 200. Training: binary cross-entropy loss, Adam optimizer, 5-fold leave-students-out cross-validation. Target AUC: ≥0.70 on held-out students."
"We track what the student knows"	"The KT model outputs a mastery vector with one probability per knowledge component. For our Data Structures course, we define 12 knowledge components: arrays, loops, conditionals, functions, pointers, recursion, linked lists, stacks, queues, trees, sorting, hashing. Each KC is mapped to 3–8 lab problems via a Q-matrix validated by the course instructor."
"It's different from existing systems"	"ClassAid (CHI 2026) is the closest existing system. It provides three hint modes (Technical, Heuristic, Auto). The mode selection is performed by the instructor manually via a dashboard. Scaffold automates this selection using DKT's mastery vector: if P(mastery) for the relevant KC < 0.40, the system selects a higher disclosure level; if > 0.70, a lower one. This is the specific technical difference."

---

7. "Improve the evaluation mechanism"

Revised evaluation criteria

What is evaluated	Metric	Target	How measured

Does KT-conditioned hinting improve learning?	Transfer-task score (Condition B vs A)	Statistically significant improvement (paired t-test, $p < 0.05$, Cohen's $d \geq 0.5$)	Unseen problem solved without AI help immediately after each condition
Does DPO beat prompt guardrails?	Leakage rate (Condition C vs B)	Condition C leakage rate < Condition B leakage rate (McNemar's test, $p < 0.05$)	Automated AST-similarity scoring of all generated hints
Is DKT more accurate than BKT?	AUC on held-out students	DKT AUC > BKT AUC (paired t-test across 5 CV folds)	Leave-students-out 5-fold cross-validation on CSEDM + collected data
Do telemetry features improve DKT?	AUC improvement of enriched DKT vs standard DKT	Statistically significant ($p < 0.05$)	Ablation study: standard features vs standard + telemetry features
Is the struggle detector accurate?	Precision and recall on "unproductive struggle" class	Precision ≥ 0.75, Recall ≥ 0.70	Hand-labeled pilot data (200+ segments labeled by two team members, inter-rater reliability reported)
Is the guardrail effective?	Leakage rate and jailbreak catch rate	Leakage rate < 5%; catch rate > 90%	AST-similarity scoring + manual review of flagged interactions

Do students feel they learned?	Perceived learning (Likert scale)	Higher in Condition B than A	Post-condition survey
Does Scaffold reduce copy-pasting?	Copy-paste incident rate	Lower in Condition B than A	Automated telemetry detection

---

8. "Refine the idea itself (after discussion with your supervisor)"

Refined one-paragraph idea statement

Scaffold is a real-time adaptive tutoring system for programming labs that solves one specific problem: existing LLM-based coding assistants give the same hint to every student regardless of what they know, because they have no model of the student's knowledge. Scaffold fixes this by maintaining a per-student, per-concept mastery estimate (via Deep Knowledge Tracing) that is updated from submission results and IDE telemetry, and injecting that estimate into the LLM's prompt context so that the generated hint is calibrated to what this specific student needs to learn next. Every hint is scored for code leakage using an AST-similarity metric (0.0–1.0) and blocked if it reveals too much of the solution. The system is evaluated through a controlled classroom study comparing KT-conditioned hinting against a standard reactive assistant, measuring whether students who receive personalized hints perform better on unseen transfer problems — the definitive test of whether learning actually occurred.

What was cut to achieve this focus

- Cross-session profiles → future work
- Copy-paste source forensics → future work

- GRPO/full RL alignment → explicitly out of scope
- Student-facing progress reports → future work

What remains (the refined scope)

1. Build a DKT model that predicts per-concept mastery from student interactions
2. Build a real-time telemetry pipeline that feeds struggle detection and KT updates
3. Build a hint engine that injects the mastery vector into LLM prompts and controls disclosure level
4. Build a leakage guardrail that scores every hint with the AST-similarity Leakage Score and blocks or regenerates hints above threshold
5. Evaluate through a controlled classroom study measuring transfer-task performance

Five things. Each one is specific, measurable, and necessary. Nothing is decorative.

Summary of Changes for Revised Proposal

Panel concern	Section affected	Change made
How does DKT work?	New Section 1 in this document; incorporated into revised proposal Section 4	Full technical explanation with equations, input/output format, and training procedure
Cold-start problem	New Section 2; incorporated into revised proposal Section 4.5	Three-layer mitigation: population prior, diagnostic quiz, rapid adaptation

Define leakage	New Section 3; incorporated into revised proposal Section 3	Formal Code Disclosure Taxonomy (L0–L4), Leakage Score metric (AST-similarity, 0.0–1.0), thresholds, and enforcement actions
Computing contribution unclear	New Section 4; incorporated into revised proposal Section 1	Four numbered contributions, each specific and verifiable
Unnecessary features	New Section 5; features cut from scope	Scope reduced from ~11 features to 7 (4 core + 3 important); 4 features deferred to future work
Explanation was qualitative, not quantitative	New Section 6; all proposal sections revised	Every claim now has a number, a metric, a threshold, or a test
Evaluation mechanism	New Section 7; incorporated into revised proposal Section 3.2	Full evaluation table with metric, target, statistical test, and measurement method for each criterion
Refine the idea	New Section 8; revised executive summary	One-paragraph refined statement; explicit list of what was cut and what remains